

# Algorithms analysis and Design

BY : DR. BASEM ALIJLA

# OUTLINE

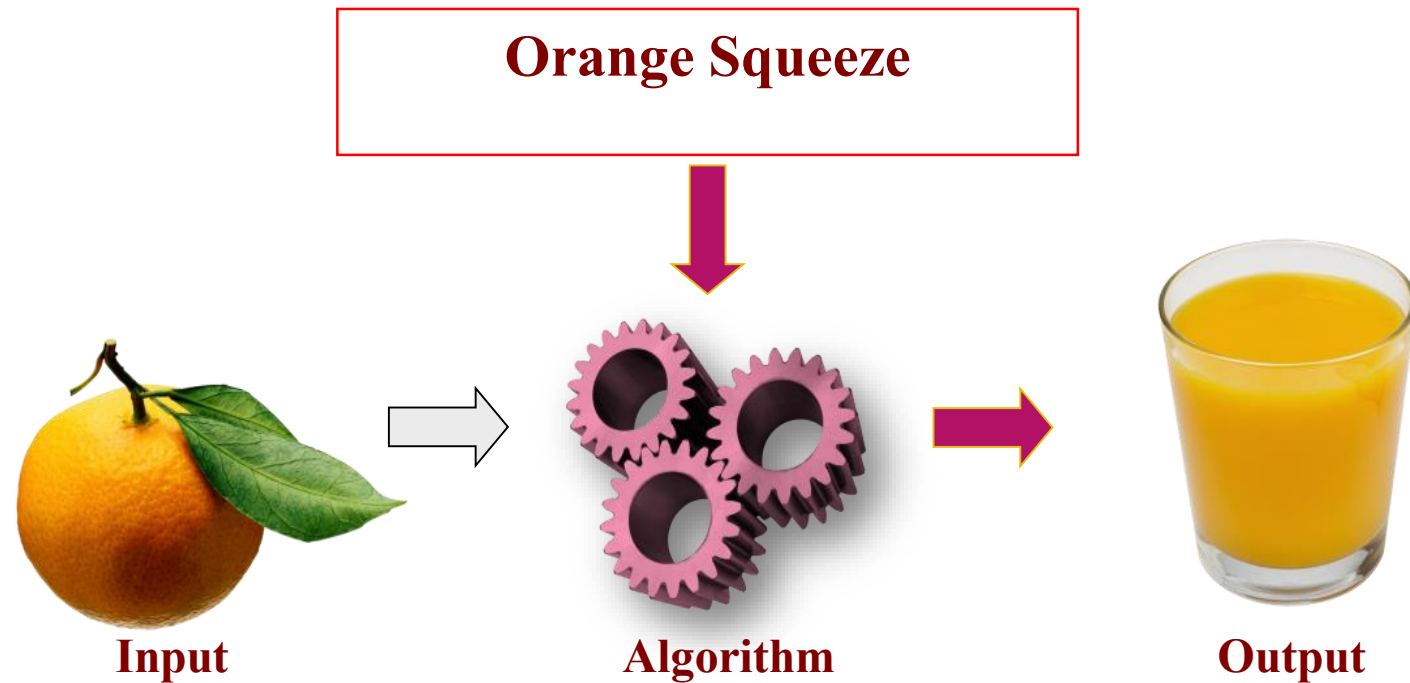
- ▶ Introduction
- ▶ Basic Concepts and Algorithmic Analysis
- ▶ Algorithm analysis and Design
- ▶ Fundamental data structures

# Analysis of Algorithms

## ► Algorithm:

a sequence of **unambiguous** instructions for solving a problem, i.e., for obtaining a required **output** for any legitimate **input** in **a finite amount of time**.

# Analysis of Algorithms



# Properties of Good Algorithm

- ▶ The non-ambiguity
- ▶ Carefully specified range of inputs
- ▶ Several algorithms for solving the same problem.
- ▶ different ideas to solve the problem with dramatically different speeds.

# OUTLINE

- ▶ Introduction
- ▶ Basic Concepts and Algorithmic Analysis
- ▶ Algorithm analysis and Design
- ▶ Fundamental data structures

# Algorithm Efficiency

- Problem: Find the sum of first  $n$  terms of a series

$$(1 + 2 + 3 + \dots + n)$$

**Algorithm 1 :**  $O(n)$

```
int sum = 0 ;  
For ( int i= 1; i <=n; i++)  
    sum += i ;
```



1



2



3

...



$n$

$O(n)$



# Algorithm Efficiency

$i = 1$  

$i = 2$   

$i = 3$    

...

$i = n$     ... 

1 2 3 ... n

**Algorithm 2 :  $O(n^2)$**

```
int sum = 0 ;  
For ( int i= 1; i <=n; i++){  
    For ( int j= 1; j <=i; j++){  
        sum +=1;  
    }  
}
```

$O(1 + 2 + \dots + n) = O(n^2)$



# Algorithm Efficiency



**Algorithm 3 :**  $O(n^2)$

```
int sum = 0 ;
For ( int i= 1; i <=n; i++){
    For ( int j= 1; j <=n; j++)
        sum += i ;
}
```

$$O(n \times n) = O(n^2)$$

# Algorithm Efficiency

**Algorithm 4** :  $O(1)$

$$\text{int sum} = \frac{n(n+1)}{2};$$

# Algorithm Efficiency

- ▶ Problem: Find the greatest common divisor of two nonnegative, not both zero integers  $m$  and  $n$
- ▶  $\text{GCD}(m, n)$  is the highest number  $a$  that divides both  $m$  and  $n$ .
- ▶ E.g.,  $\text{GCD}(60, 24) = 12$ ,  $\text{GCD}(60, 0) = 60$

# Algorithms to find the GCD

## ► Consecutive integer checking algorithm

**Step 1** Assign the value of  $\min(m, n)$  to  $t$

**Step 2** Divide  $m$  by  $t$ . If the remainder is 0, go to Step 3;  
otherwise, go to Step 4

**Step 3** Divide  $n$  by  $t$ . If the remainder is 0, return  $t$  and stop;  
otherwise, go to Step 4

**Step 4** Decrease  $t$  by 1 and go to Step 2

**$O(n)$**

# Euclid's Algorithms to find GCD

- ▶ Euclid's Algorithm : is based on repeated application of equality  $\text{GCD}(m,n) = \text{GCD}(n, m \bmod n)$  until the second number becomes 0.

Example:

$$\begin{aligned}\text{GCD}(60,24) \\ &= \text{GCD}(24,12) \\ &= \text{GCD}(12,0) = \mathbf{12}\end{aligned}$$

# Euclid's Algorithm to find GCD

**Step 1:** - If  $n = 0$ , return  $m$  and stop; otherwise go to Step 2

**Step 2:-** Divide  $m$  by  $n$  and assign the value for the remainder to  $r$

**Step 3:-** Assign the value of  $n$  to  $m$  and the value of  $r$  to  $n$ .

Go to Step 1.

**$O(\log n)$**

```
while  $n \neq 0$  do  
     $temp = m \bmod n$   
     $m = n$   
     $n = temp$   
return  $m$ 
```

# Main Issues related to algorithms

## ► Two main issues related to algorithms

1. How to design algorithms
2. How to analyze algorithm efficiency

# Algorithms and Problem Solving

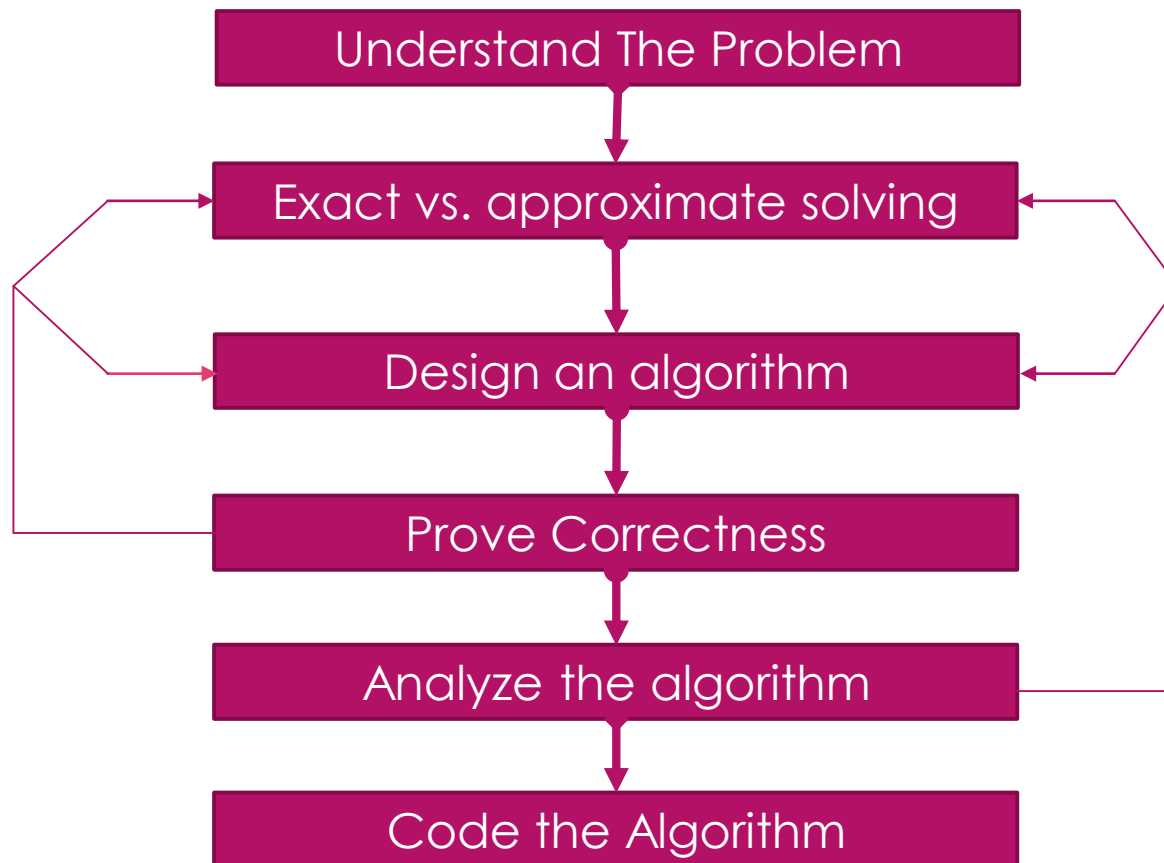
- ▶ Consider algorithms to be procedural solutions to problems
- ▶ Algorithm design and analysis process



# OUTLINE

- ▶ Introduction
- ▶ Basic Concepts and Algorithmic Analysis
- ▶ Algorithm analysis and Design
- ▶ Fundamental data structures

# Algorithm analysis and Design



# Understand The Problem

- ▶ Read the problem's description carefully.
- ▶ use a known algorithm for solving it
- ▶ Design your own algorithm if not find a available algorithm.
- ▶ An input to an algorithm specifies an ***instance*** of the problem the algorithm solves.
- ▶ **Sequential algorithm or parallel algorithms**

# Exact or approximate problem

## ► Exact Algorithm

## ► Approximate Algorithm

- Problem cannot be solved exactly
- Exact solution is computationally expensive

# Design Algorithm

- ▶ **What is an algorithm design technique ?  
(strategy paradigm).**
- ▶ Is a general approach for solving problems algorithmically applicable for various problem.

# Algorithm design strategies

- ❖ **Brute force**
- ❖ **Divide and conquer**
- ❖ Decrease and conquer
- ❖ Transform and conquer
- ❖ Space and time tradeoffs
- ❖ **Greedy approach**
- ❖ **Dynamic programming**
- ❖ Iterative improvement
- ❖ Backtracking
- ❖ Branch and bound

# Proving an Algorithm's Correctness

- ▶ Prove the correctness of the algorithm for all possible input in a finite amount of time.
- ▶  $\text{GCD}(m, n) = \text{GCD}(n, m \bmod n)$ .
- ▶  $m \bmod n$  will become zero.
- ▶ Prove Correctness using mathematical induction
- ▶ Prove incorrectness by only one incorrect example.

# Analysis of algorithms

## □ How good is the algorithm?

- time efficiency
- Space efficiency
- Simplicity
- Generality ( problem or input).

## □ Does there exist a better algorithm?

- Lower bounds
- Optimality



# Important problem types

## ▶ **Sorting**

- stable sorting algorithm
- In-place – no Extra memory

## ▶ **Searching**

## ▶ **String processing**

- String matching

## ▶ **graph problems**

- Graph traversal & Shortest path
- TSP algorithms (NP hard)

## ▶ **Combinatorial problems**

- e.g., TSP and Graph coloring
- Find the combinatorial objects

## ▶ **Geometric problems**

- geometric objects such as points, lines, and polygons.
- The closest-pair problem
- convex-hull problem

## ▶ **Numerical problems**

- Problems that involve mathematical objects of continuous nature

# OUTLINE

- ▶ Introduction
- ▶ Basic Concepts and Algorithmic Analysis
- ▶ Algorithm analysis and Design
- ▶ Fundamental data structures

# Fundamental data structures

- **Data Structure** : a particular scheme of organizing related data items.

**Algorithm + Data Structure = Program**

- ❖ list
- ❖ array
- ❖ linked list
- ❖ string
- ❖ stack
- ❖ queue
- ❖ priority queue
- ❖ graph
- ❖ tree
- ❖ set and dictionary

# Use appropriate Data Structure

► Array : **Find ( 3 )**



► Find 3 with 1 compare process ( Best case )

# Use appropriate Data Structure

► Array : **Find ( 8 )**



► Find 3 with 1 compare process

# Use appropriate Data Structure

► Array : **Find ( 8 )**



► Find 3 with 1 compare process

# Use appropriate Data Structure

- ▶ Array : **Find ( 8 )**



- ▶ Find 3 with 1 compare process
- ▶ Find 8 with 3 compare process ( average case)

# Use appropriate Data Structure

► Array : **Find ( 12 )**



► Find 3 with 1 compare process

► Find 8 with 3 compare process



# Use appropriate Data Structure

► Array : **Find ( 12 )**

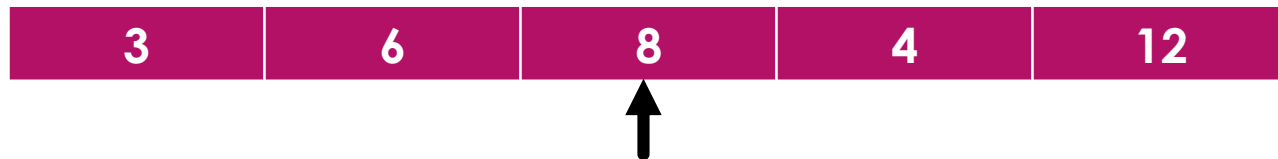


► Find 3 with 1 compare process

► Find 8 with 3 compare process

# Use appropriate Data Structure

► Array : **Find ( 12 )**



► Find 3 with 1 compare process

► Find 8 with 3 compare process

# Use appropriate Data Structure

► Array : **Find ( 12 )**



► Find 3 with 1 compare process

► Find 8 with 3 compare process

# Use appropriate Data Structure

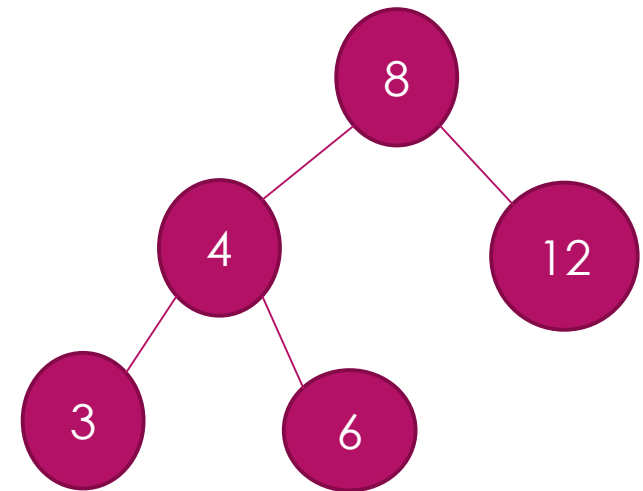
## ► Array :Find ( 12 )



- Find 3 with 1 compare process
- Find 8 with 3 compare process
- Find 12 with n compare process ( worst case)

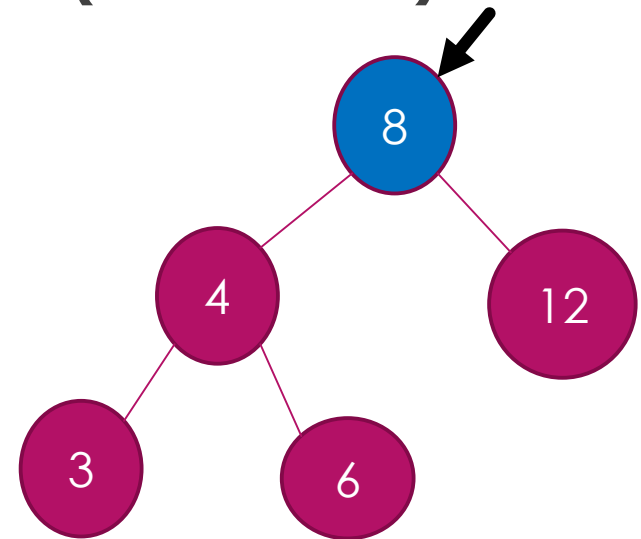
# Use appropriate Data Structure

## ► Binary Search Tree



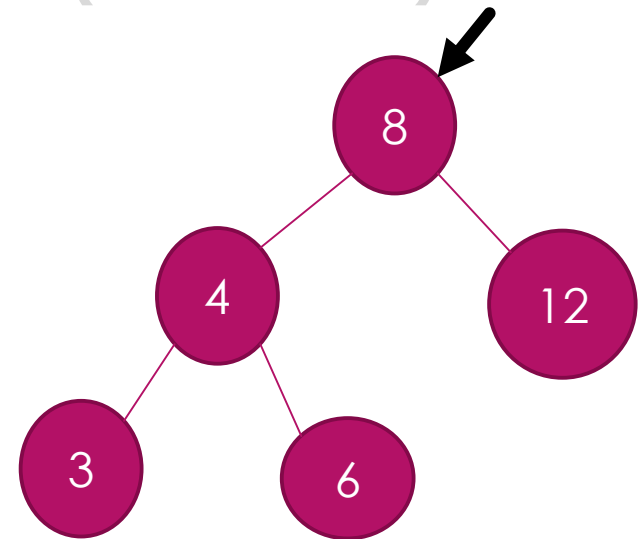
# Use appropriate Data Structure

- ▶ Binary Search Tree Find ( 8 )
- ▶ Find 8 with 1 compare process ( best case)



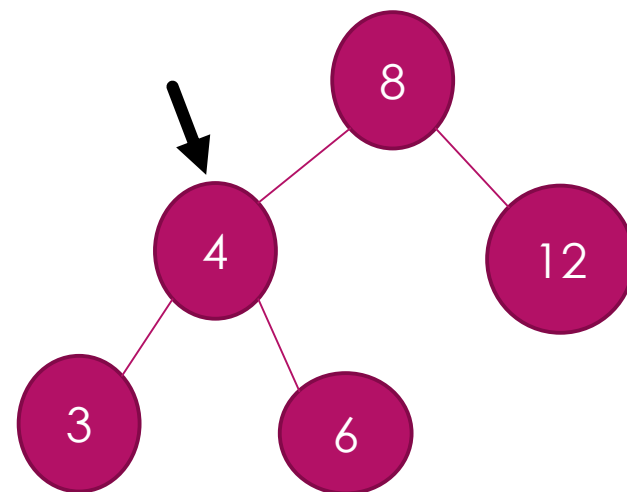
# Use appropriate Data Structure

- ▶ **Binary Search Tree Find ( 3 )**
- ▶ Find 8 with 1 compare process ( best case)



# Use appropriate Data Structure

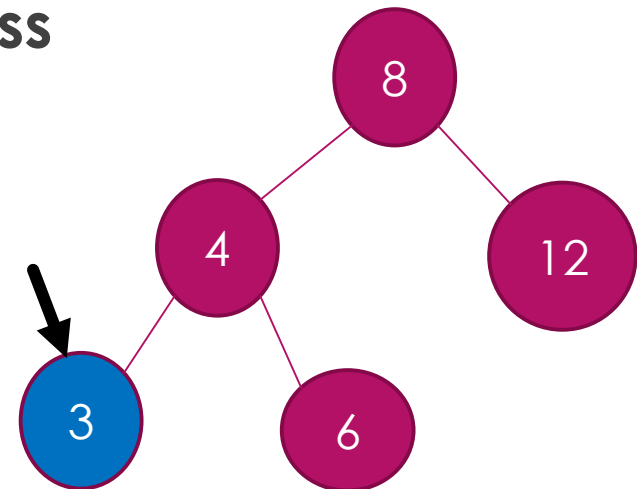
- ▶ **Binary Search Tree Find ( 3 )**
- ▶ Find 8 with 1 compare process ( best case)





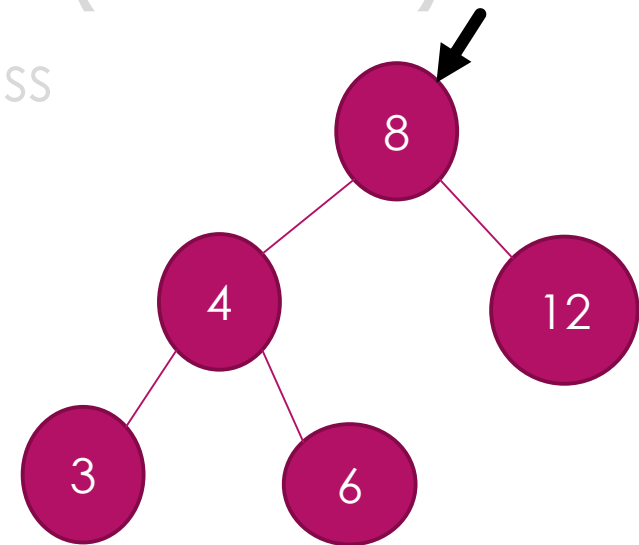
# Use appropriate Data Structure

- ▶ **Binary Search Tree Find ( 3 )**
- ▶ Find 8 with 1 compare process ( best case)
- ▶ Find 3 with 3 compare process



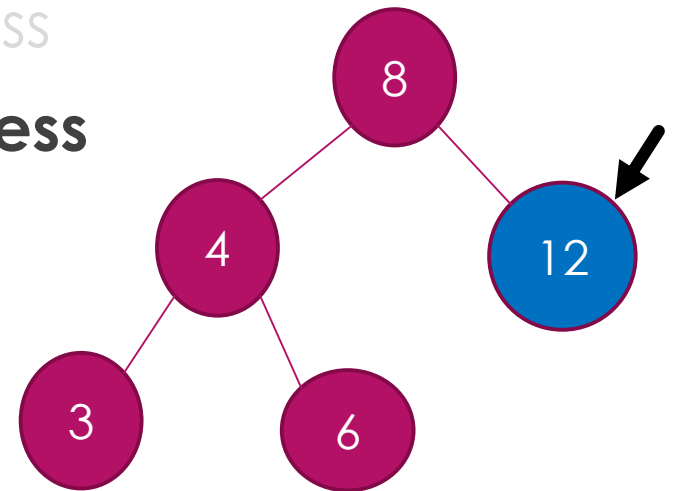
# Use appropriate Data Structure

- ▶ **Binary Search Tree Find ( 12 )**
- ▶ Find 8 with 1 compare process ( best case)
- ▶ Find 3 with 3 compare process



# Use appropriate Data Structure

- ▶ **Binary Search Tree Find ( 12 )**
- ▶ Find 8 with 1 compare process ( best case)
- ▶ Find 3 with 3 compare process
- ▶ **Find 12 with 2 compare process**

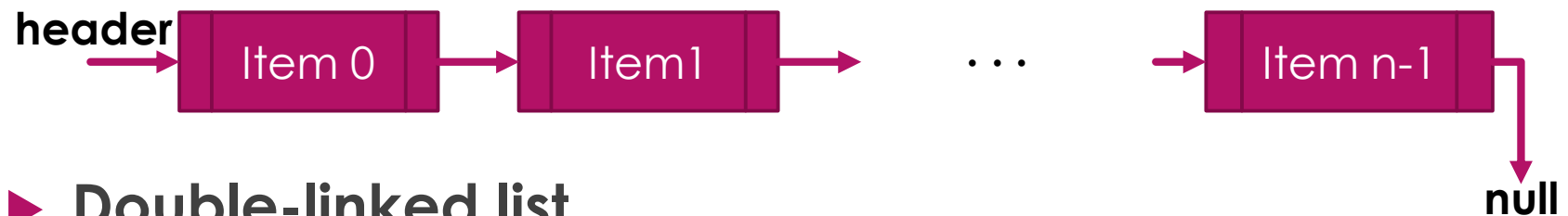


# Linear Data Structure

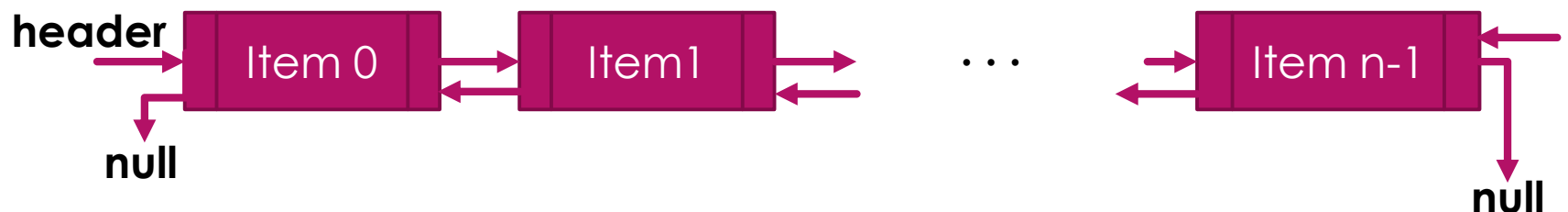
- **Array** : is a sequence of  $n$  items of the same data type



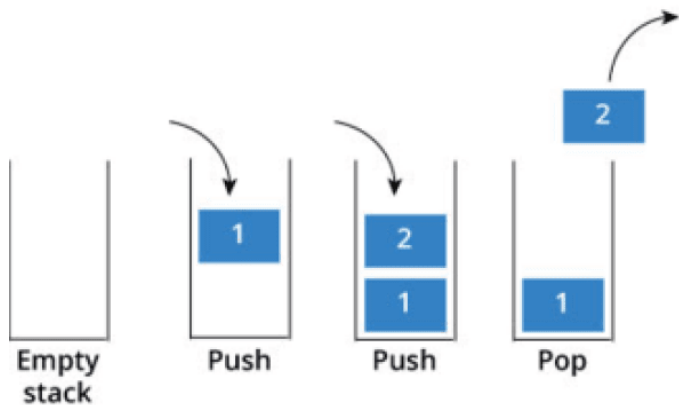
- **Single-linked list**



- **Double-linked list**



# Linear Data Structure



**Stack**  
FILO



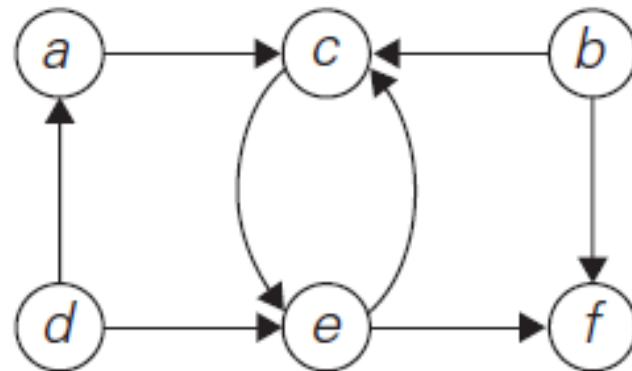
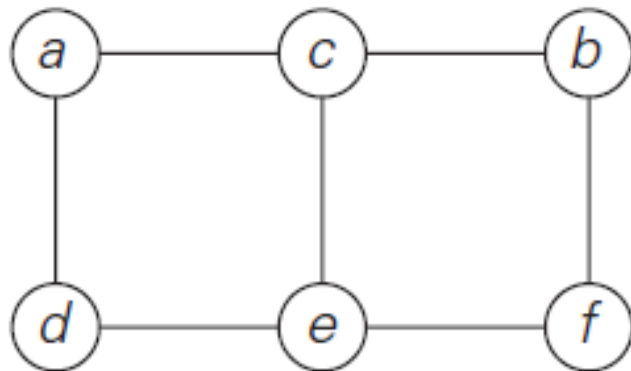
**Queue**  
FIFO

# Non Linear Data Structure

- ▶ Tree
- ▶ Graph

# Graph Terminology

- **Graphs** : collection of nodes connected by edges  $G = (V, E)$ 
  - Set of vertices ( $V$ ) , Set of Edges ( $E$ )
  - Undirected Graph :  $(u,v) = (v,u)$
  - Directed Graph or digraphs :  $(u,v) \neq (v,u)$



# Graph Terminology

## ► Graphs representation

- Adjacency matrix : (symmetric and asymmetric)
- Adjacency list

|          | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i> | <i>f</i> |
|----------|----------|----------|----------|----------|----------|----------|
| <i>a</i> | 0        | 0        | 1        | 1        | 0        | 0        |
| <i>b</i> | 0        | 0        | 1        | 0        | 0        | 1        |
| <i>c</i> | 1        | 1        | 0        | 0        | 1        | 0        |
| <i>d</i> | 1        | 0        | 0        | 0        | 1        | 0        |
| <i>e</i> | 0        | 0        | 1        | 1        | 0        | 1        |
| <i>f</i> | 0        | 1        | 0        | 0        | 1        | 0        |

|          |   |          |   |          |            |
|----------|---|----------|---|----------|------------|
| <i>a</i> | → | <i>c</i> | → | <i>d</i> |            |
| <i>b</i> | → | <i>c</i> | → | <i>f</i> |            |
| <i>c</i> | → | <i>a</i> | → | <i>b</i> | → <i>e</i> |
| <i>d</i> | → | <i>a</i> | → | <i>e</i> |            |
| <i>e</i> | → | <i>c</i> | → | <i>d</i> | → <i>f</i> |
| <i>f</i> | → | <i>b</i> | → | <i>e</i> |            |



# Graph Terminology

## ► **Complete Graph:**

- A graph with every pair of its vertices connected by an edge.
- Denoted as  $K_{|V|}$

## ► **Dense :**

- A graph with relatively few possible edges missing

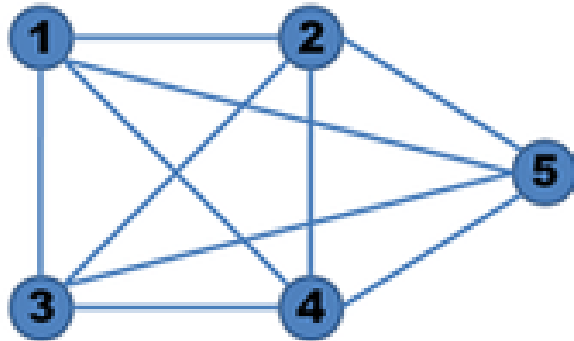
## ► **Sparse :**

- a graph with few edges relative to the number of its vertices

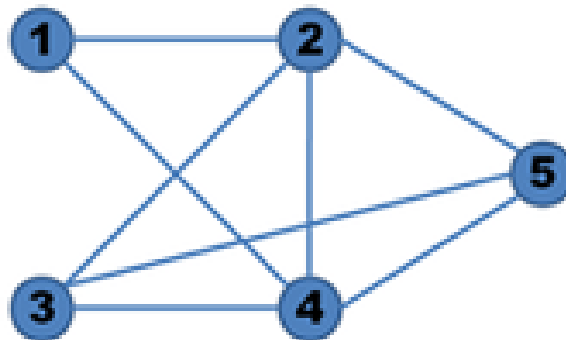
# Graph Terminology

5.

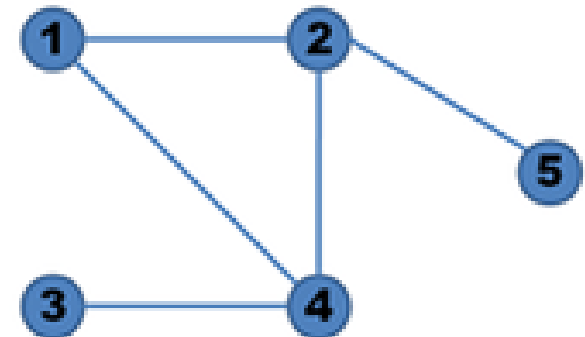
Complete Graph



Dense Graph

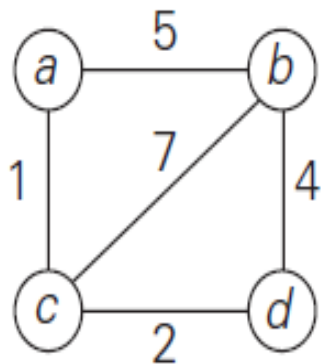


Sparse Graph



# Graph Terminology

- **Weighted Graphs:** is a graph with numbers assigned to its edges called weight or cost.

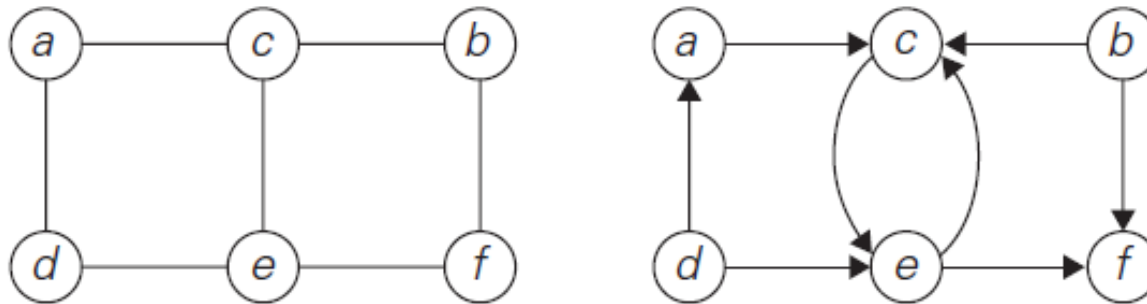


|          | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> |
|----------|----------|----------|----------|----------|
| <i>a</i> | $\infty$ | 5        | 1        | $\infty$ |
| <i>b</i> | 5        | $\infty$ | 7        | 4        |
| <i>c</i> | 1        | 7        | $\infty$ | 2        |
| <i>d</i> | $\infty$ | 4        | 2        | $\infty$ |

| <i>a</i> | $\rightarrow b, 5 \rightarrow c, 1$                  |
|----------|------------------------------------------------------|
| <i>b</i> | $\rightarrow a, 5 \rightarrow c, 7 \rightarrow d, 4$ |
| <i>c</i> | $\rightarrow a, 1 \rightarrow b, 7 \rightarrow d, 2$ |
| <i>d</i> | $\rightarrow b, 4 \rightarrow c, 2$                  |

# Graph Terminology

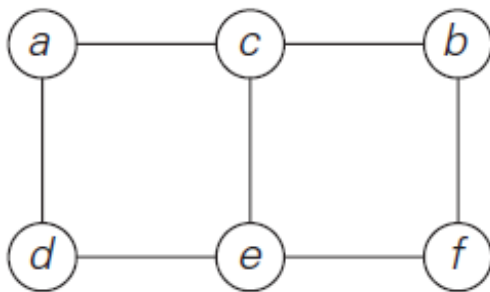
- **Weighted Graphs:** is a graph with numbers assigned to its edges called weight or cost.



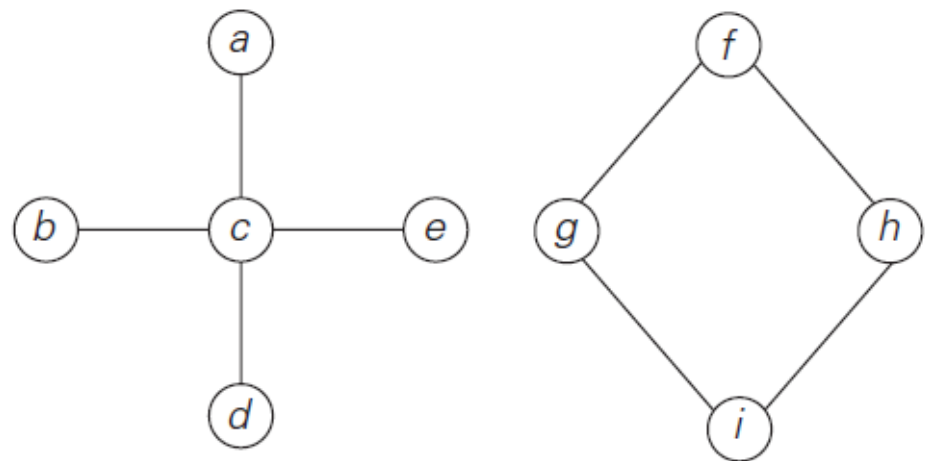
- Simple path  $a \rightarrow c \rightarrow b \rightarrow f$  of length 3
- Not simple path  $a \rightarrow c \rightarrow e \rightarrow c \rightarrow b \rightarrow f$  of length 5
- Directed path :  $a \rightarrow c \rightarrow e \rightarrow e$

# Graph Terminology

- ▶ **Connected Graphs (single piece):** if for every pair of its vertices  $u$  and  $v$  there is a path from  $u$  to  $v$ .
- ▶ **Not connected graph ( multiple piece):**



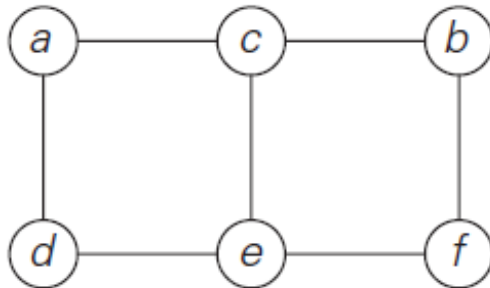
**Connected Graph**



**Not Connected Graph**

# Graph Terminology

- ▶ A **cycle** is a path of a positive length that starts and ends at the same vertex and does not traverse the same edge more than once.

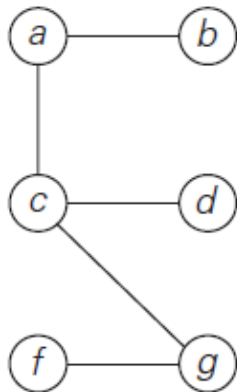


$a \rightarrow c \rightarrow e \rightarrow d \rightarrow a$  is cycle

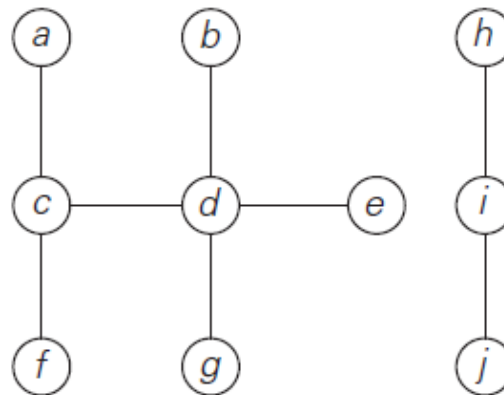
- ▶ A graph with no cycles is said to be **acyclic**.

# Tree Terminology

- ▶ A **tree** (more accurately, a **free tree**) is a connected acyclic graph.
- ▶ **Forest** : acyclic graph not necessary to be connected.



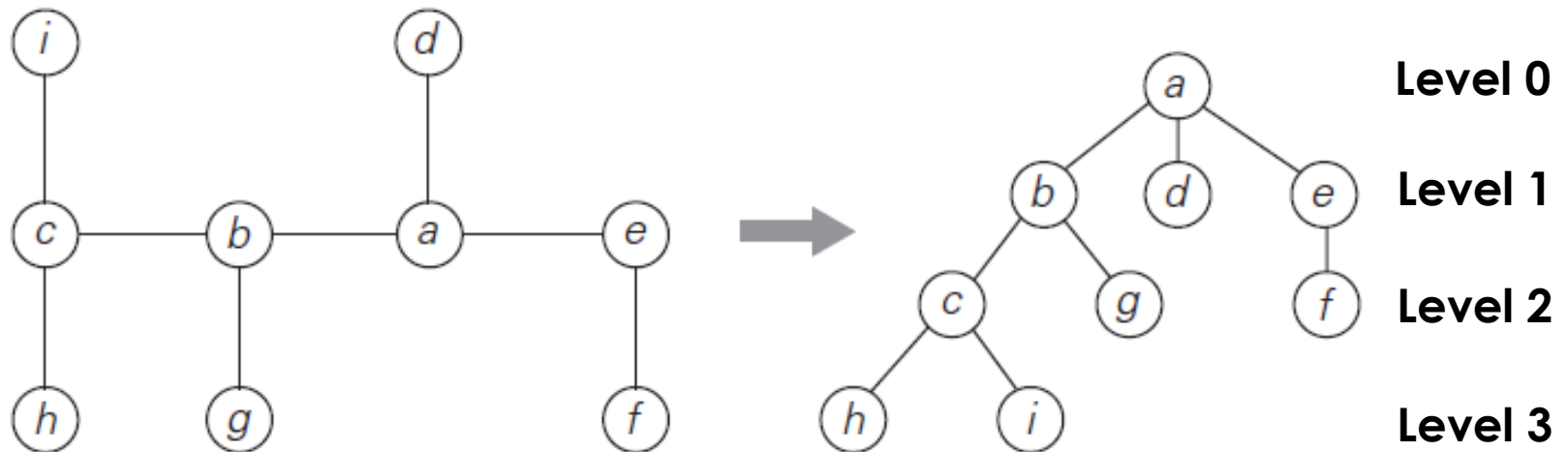
**Tree**



**Forest**

# Tree Terminology

## ► A Rooted *tree*



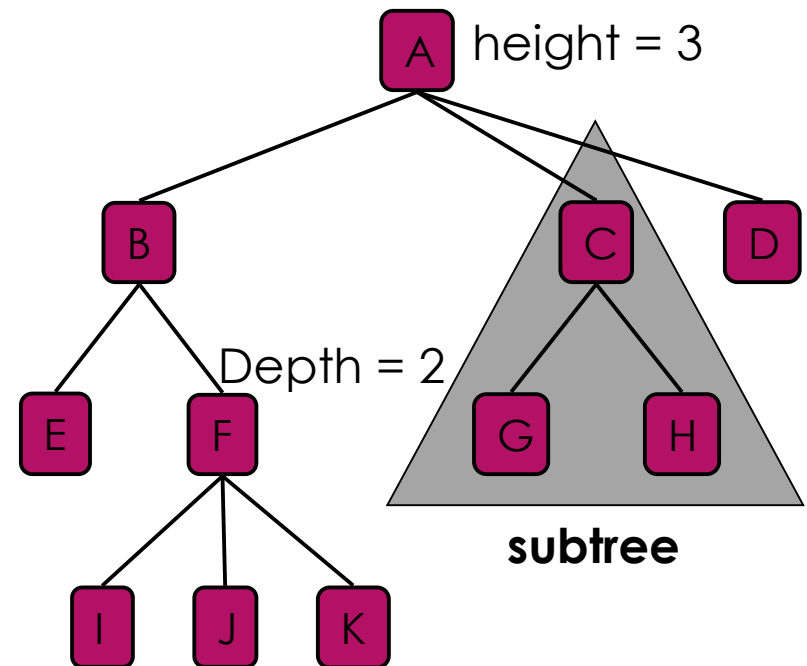


# Tree Terminology

- ▶ **Root**
- ▶ **Parent**
- ▶ **Child**
- ▶ **Ancestors:** all the vertices on the simple path from the root to that vertex.
- ▶ **Descendants:** All the vertices for which a vertex  $v$  is an ancestor
- ▶ **Siblings:** vertices that have the same parent
- ▶ **Subtree**
- ▶ **Depth of node  $v$ :** is the length of the simple path from the **root** to  $v$ .
- ▶ **Height of node  $v$ :** is the length of the longest simple path from the node  $v$  to a **leaf**

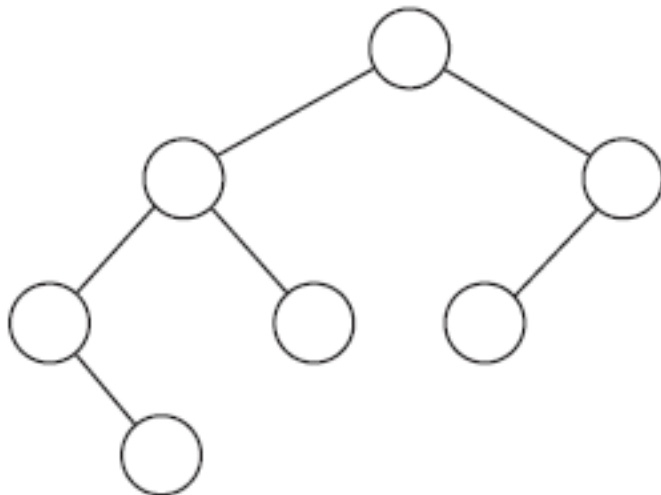
# Tree Terminology

- ▶ **Root:** A
- ▶ **Internal node:** (A, B, C, F)
- ▶ **leaf node:** (E, I, J, K, G, H, D)
- ▶ **Ancestors of k** are K,F,B,A
- ▶ **Descendants of B** are B,F,I,J,K
- ▶ **Siblings:** B , C and D
- ▶ Height of tree = 3
- ▶ Depth of F = 2

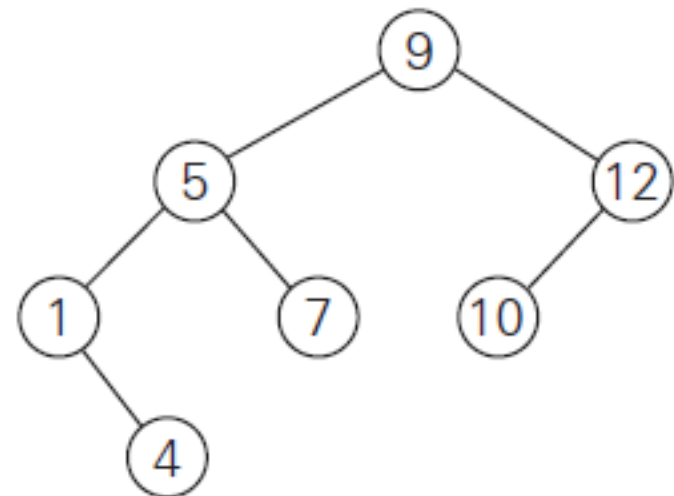


# Tree Terminology

- ▶ A **binary tree** an ordered tree in which every vertex has at most two children (left and right)
- ▶ **binary search trees (BST)**



**Binary Tree**



**Binary Search Tree (BST)**